

Mastering Advanced Policy Gradients & PPO

Complete Mathematical & Algorithmic Foundations: Policy Collapse, Kakade-Langford Monotonic Improvement Guarantee, Natural Policy Gradients, Fisher Information Matrix, TRPO, Clipped Surrogate PPO, and Isaac Lab Scaling

Instructor: Prof. Sergey Levine **Curriculum:** Berkeley CS285 + Schulman et al. (TRPO/ PPO) + Achiam (Spinning Up) **Scope:** General Trust Region Optimization & Robotics Slicing

0. The Executive Mental Map: Why Does Lecture 10 Exist?

In Lecture 5 and Lecture 6, we derived the Policy Gradient Theorem and learned how to calculate policy updates via gradient ascent: $\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$.

The Fatal Flaw of Standard Gradient Steps: In supervised learning, taking a step with a learning rate that is slightly too large causes a temporary bump in test error; the next mini-batch corrects it. **In reinforcement learning, the policy generates its own future training data.**

If a single oversized gradient step pushes the policy into a destructive regime (e.g., slamming the knife into the cutting table or pitching a drone upside down), **every rollout collected in the next iteration will fail completely.** The neural network receives zero reward across the entire batch, variance explodes, and the policy enters an unrecoverable catastrophic collapse.

Lecture 10 introduces Trust Region Policy Optimization (TRPO) and Proximal Policy Optimization (PPO): the algorithms that place a rigorous mathematical "safety leash" on policy updates, guaranteeing monotonic policy improvement and enabling reliable robot learning in massively parallel simulators.

1. Policy Collapse

Why Bad Steps
Cause Death Spirals

2. Monotonic Bounds

Kakade & Langford Proof
Surrogate Lower Bound

3. Natural Gradient

Fisher Matrix F
Conjugate Gradients

4. PPO-Clip Engine

$\min(rA, \text{clip}(r)A)$
The 4-Quadrant Analysis

5. Parallel Scaling

Isaac Lab 4096 Envs
Epoch Reuse Paradigm

Part 1: The Catastrophe of Policy Collapse

(Slides 1–15) To understand why standard gradient descent fails in RL, we compare how error recovery functions in Supervised Learning vs. Reinforcement Learning:

Supervised Learning (Stable)

Fixed offline dataset (ImageNet)
 Oversized Step → Bad batch loss
 Next Batch: Recovers easily
 Data distribution remains static!

Reinforcement Learning (FRAGILE)

Policy collects its own training data!
 Oversized Step → Destructive Actions
 Next Batch: 100% Failed Trajectories
 CATASTROPHIC COLLAPSE (Irrecoverable)

1.1 The State Distribution Shift Dilemma

When updating policy parameters from θ_{old} to θ , the true expected performance of the new policy is:

$$J(\pi_\theta) = \mathbb{E}_{s \sim d^{\pi_\theta}(s)} [\mathbb{E}_{a \sim \pi_\theta(a|s)} [Q^{\pi_{\text{old}}}(s, a)]]$$

In practice, we evaluate actions using states sampled from the **old policy**: $s \sim d^{\pi_{\text{old}}}(s)$. We are ignoring the fact that changing the policy alters the state visitation distribution!

Sergey Levine's Core Question: "When is it mathematically valid to approximate $d^{\pi_\theta}(s)$ with $d^{\pi_{\text{old}}}(s)$?"

The Answer: Only when the probability distributions $\pi_\theta(a|s)$ and $\pi_{\text{old}}(a|s)$ remain extremely close in distribution space, bounded by a **Trust Region**!

Part 2: The Kakade-Langford Monotonic Improvement Guarantee

(Slides 16–35) In their 2002 paper, Sham Kakade and John Langford proved the fundamental theorem underpinning all modern trust region methods:

THE EXACT POLICY VALUE IDENTITY

For any two arbitrary policies π and $\tilde{\pi}$:

$$J(\tilde{\pi}) = J(\pi) + \mathbb{E}_{\tau \sim \tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi}(s_t, a_t) \right] = J(\pi) + \sum_s d^{\tilde{\pi}}(s) \sum_a \tilde{\pi}(a | s) A^{\pi}(s, a)$$

Proof: Express advantage as $A^{\pi}(s_t, a_t) = r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$. The infinite sum telescopes:

$$\sum_{t=0}^{\infty} \gamma^t (r_t + \gamma V^{\pi}(s_{t+1}) - V^{\pi}(s_t)) = \sum_{t=0}^{\infty} \gamma^t r_t - V^{\pi}(s_0)$$

Taking expectations with respect to $\tilde{\pi}$:

$$\mathbb{E}_{\tilde{\pi}} \left[\sum_{t=0}^{\infty} \gamma^t A^{\pi}(s_t, a_t) \right] = J(\tilde{\pi}) - \mathbb{E}_{s_0} [V^{\pi}(s_0)] = J(\tilde{\pi}) - J(\pi)$$

2.1 The Surrogate Lower Bound

Because the true state distribution $d^{\tilde{\pi}}(s)$ is unknown, we define the **Surrogate Objective** using the old state distribution:

$$L_{\pi}(\tilde{\pi}) = J(\pi) + \sum_s d^{\pi}(s) \sum_a \tilde{\pi}(a | s) A^{\pi}(s, a)$$

The TRPO Monotonic Lower Bound (Schulman et al., 2015):

$$J(\tilde{\pi}) \geq L_{\pi}(\tilde{\pi}) - C \cdot D_{\text{KL}}^{\max}(\pi, \tilde{\pi})$$

$$\text{where } C = \frac{4\epsilon\gamma}{(1-\gamma)^2} \quad \text{and} \quad D_{\text{KL}}^{\max}(\pi, \tilde{\pi}) = \max_s D_{\text{KL}}(\pi(\cdot | s) \| \tilde{\pi}(\cdot | s))$$

The Monotonic Improvement Guarantee: If we maximize the right-hand side, we are guaranteed that true performance $J(\tilde{\pi})$ will monotonically increase at every iteration!

Part 3: From Natural Policy Gradients & TRPO to PPO

(Slides 36–52) How do we optimize within a trust region in practice?

3.1 Natural Policy Gradient & The Fisher Information Matrix

Standard gradient descent steps in parameter Euclidean space: $\|\Delta\theta\|^2 \leq \epsilon$. But in neural networks, a tiny shift in one layer's weight can cause a massive shift in output probabilities, while a large shift in another weight causes zero change! **Natural Policy Gradients** measure distance directly in probability distribution space using the **Fisher Information Matrix (FIM)** F :

$$F(\theta) = \mathbb{E}_{s \sim d^\pi, a \sim \pi} [\nabla_\theta \log \pi_\theta(a | s) \nabla_\theta \log \pi_\theta(a | s)^T]$$

$$\Delta\theta_{\text{Natural}} \propto F^{-1} \nabla_\theta J(\theta)$$

3.2 TRPO: Trust Region Policy Optimization

TRPO solves the constrained optimization problem:

$$\max_{\theta} L_{\theta_{\text{old}}}(\theta) \quad \text{subject to} \quad \bar{D}_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_\theta) \leq \delta$$

Using second-order Taylor expansion on the constraint and first-order on the objective, TRPO solves:

$$\Delta\theta = \sqrt{\frac{2\delta}{g^T F^{-1} g}} F^{-1} g$$

where $g = \nabla_\theta L$. To avoid inverting the massive $P \times P$ matrix F directly, TRPO uses the **Conjugate Gradient (CG)** algorithm and **Fisher-Vector Products** (Fv).

THE DOWNSIDE OF TRPO: WHY ROBOTICS MOVED TO PPO

While TRPO is theoretically elegant, Conjugate Gradient requires multiple backpropagation passes per step to compute Fisher-vector products. It cannot be combined with first-order optimizers like Adam, struggles with recurrent networks, and is computationally prohibitive when scaling to 4,096 parallel GPU simulation environments in Isaac Lab.

Part 4: Proximal Policy Optimization (PPO-Clip) Deconstructed

(Slides 53–70 & Spinning Up ch16) In 2017, John Schulman, Sergey Levine et al. introduced PPO to retain TRPO's monotonic stability while using simple, first-order gradient descent.

4.1 The Clipped Surrogate Objective

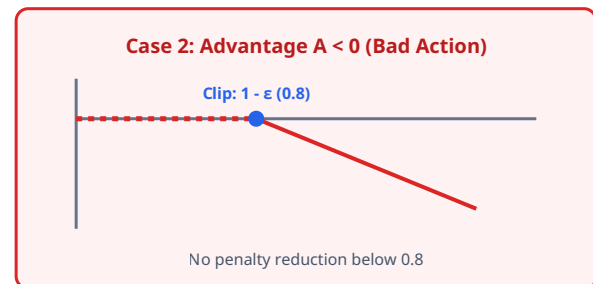
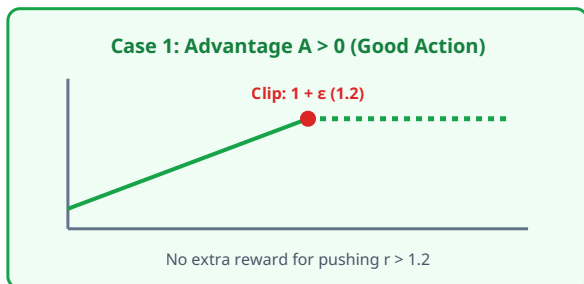
Define the probability ratio:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

The PPO Clipped Surrogate Objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip} \left(r_t(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t \right) \right]$$

4.2 The 4-Quadrant Analysis of PPO Clipping



1. **Positive Advantage** ($A > 0$), **Ratio** $r \leq 1 + \epsilon$: Normal policy gradient. The action was good, so we increase its probability.
2. **Positive Advantage** ($A > 0$), **Ratio** $r > 1 + \epsilon$: The objective is **clipped** to $(1 + \epsilon)A$. Gradient $\frac{\partial}{\partial \theta} = 0$. The optimizer is prevented from excessively over-committing to this move!
3. **Negative Advantage** ($A < 0$), **Ratio** $r \geq 1 - \epsilon$: Normal policy gradient. The action was bad, so we decrease its probability.
4. **Negative Advantage** ($A < 0$), **Ratio** $r < 1 - \epsilon$: The objective is **clipped** to $(1 - \epsilon)A$. Gradient $\frac{\partial}{\partial \theta} = 0$. The optimizer is prevented from over-penalizing an already suppressed action!

Part 5: Production Engineering with Isaac Lab & SkRL

```
# Complete Vectorized PPO Loss Function in PyTorch
import torch
import torch.nn.functional as F

def compute_ppo_loss(actor, critic, obs, actions, log_prob_old, returns, advantages, clip_eps=0.2, c_v=0):
    # 1. Evaluate current policy on historical batch
    dist = actor(obs)
    log_prob_new = dist.log_prob(actions).sum(dim=-1, keepdim=True)
    entropy = dist.entropy().sum(dim=-1, keepdim=True).mean()

    # 2. Probability Ratio r_t(theta)
    ratio = torch.exp(log_prob_new - log_prob_old)

    # 3. Clipped Surrogate Policy Loss
    surr1 = ratio * advantages
    surr2 = torch.clamp(ratio, 1.0 - clip_eps, 1.0 + clip_eps) * advantages
    actor_loss = -torch.min(surr1, surr2).mean()

    # 4. Value Loss (MSE with optional clipping)
    values = critic(obs)
    critic_loss = c_v * F.mse_loss(values, returns)

    # 5. Composite Loss
    total_loss = actor_loss + critic_loss - c_ent * entropy
    return total_loss, actor_loss, critic_loss, entropy
```

THE LOG-PROB SUBTRACTION VS RATIO TRAP

Never compute the probability ratio via division: ``ratio = pi_new / pi_old``. Probabilities in continuous spaces or high-dimensional token spaces evaluate to microscopic values like 10^{-45} , causing immediate division by zero and ``NaN``. Always compute log probabilities and exponentiate their difference:

```
ratio = torch.exp(log_prob_new - log_prob_old)
```

Part 6: Interactive Tablet Self-Test Quiz

Question 1: In PPO, what happens when an action yields a positive advantage ($A > 0$) and its probability ratio reaches $r_t(\theta) = 1.35$ (with $\epsilon = 0.2$)?

Answer: The ratio 1.35 exceeds the upper clip boundary $1 + \epsilon = 1.2$. The objective term is clipped to $1.2 \times A$, causing the gradient with respect to policy weights θ to drop to exactly zero. This prevents the optimizer from aggressively over-committing to this single action.

Question 2: Why did OpenAI abandon PPO-Penalty (adaptive KL penalty) in favor of PPO-Clip?

Answer: PPO-Penalty requires continuous manual heuristics to scale the penalty coefficient β . If β is tuned incorrectly, updates become either too conservative (freezing learning) or too aggressive (causing collapse). PPO-Clip enforces the trust region boundary directly in the objective with a fixed scalar $\epsilon = 0.2$, requiring zero hyperparameter tuning.

Question 3: Why does `value\_clip: 0.2` matter when the knife breaks through the tomato skin?

Answer: Cuticle rupture causes an abrupt drop in contact forces and a surge in rewards. Without value clipping, the Critic's loss would spike, causing a massive gradient update that distorts value predictions for all non-contact states. Value clipping restricts $V(s)$ updates to within ± 0.2 of previous estimates, preserving value stability.

Part 7: Thesis Defense Master Cheatsheet (Lecture 10 Focus)

Q1: "WHAT IS POLICY COLLAPSE, AND HOW DOES PPO PREVENT IT DURING TRAINING?"

Answer: "Policy collapse occurs in reinforcement learning when an oversized gradient step alters policy behavior such that all future rollouts result in failure. Because the policy collects its own training data, the replay buffer is corrupted with catastrophic trajectories, making recovery impossible. PPO prevents this by clipping the probability ratio $r_t(\theta) = \pi_\theta / \pi_{\text{old}}$ within $[1 - \epsilon, 1 + \epsilon]$ ($\epsilon = 0.2$). This bounds the policy update to a local trust region, mathematically preventing destructive updates."

Q2: "WHY CAN PPO PERFORM MULTIPLE TRAINING EPOCHS ON THE SAME ROLLOUT BATCH, WHILE REINFORCE CANNOT?"

Answer: "REINFORCE assumes on-policy data sampled strictly from the current policy distribution. As soon as a single gradient step is taken, the policy changes, rendering the old rollout data off-policy and invalid. PPO incorporates an importance-sampling ratio $r_t(\theta)$ combined with surrogate clipping. This enables the algorithm to safely execute 4 to 8 gradient epochs on the same rollout buffer without causing policy divergence, dramatically increasing sample efficiency."

Q3: "WHY CHOOSE PPO OVER TRPO FOR TRAINING IN ISAAC LAB?"

Answer: "TRPO strictly enforces the trust region via a hard KL constraint ($\mathbb{E}[D_{\text{KL}}] \leq \delta$), which requires computing the Fisher Information Matrix and solving a quadratic program using Conjugate Gradients. This is computationally expensive and difficult to parallelize on GPUs. PPO achieves equivalent empirical stability through a simple first-order clipped surrogate objective that integrates seamlessly with standard GPU optimizers like Adam."